

WEB DEVELOPMENT • 3RD SEMESTER

Week 01 • Block 0

Welcome to Web Development

<web-development> • Hochschule Ansbach

Matthias Kellnhofer

About me



Matthias Kellnhofer

Software Engineer
Regensburg

Work

- **Full Stack Developer** at Dracoon (*since 2015*) — Java (Spring) backend dev; Android Java+Kotlin dev; lead of the apps team
- **SAP BI / Web Developer** at Osram (*2011–2014*)

Education

- **M.Sc. Computer Science** (part-time), OTH Regensburg (*2015*)
- **B.Sc. Computer Science**, OTH Regensburg (*2010*)

PART 1

Where Is the Web?

A quick quiz

Built with web technologies?

Product	What it is	Web?
hs-ansbach.de	Our university's website	?
VS Code	Code editor — we use it in this course	?
Spotify	Music app for the desktop	?
Steam	Game store and launcher for the desktop	?
Claude Code	AI coding assistant in the terminal	?

Web technologies = the user interface is built with **HTML, CSS and JavaScript**, or with a framework that uses these.

All of them

Product	Web?	How
hs-ansbach.de	✓	HTML, CSS and JavaScript, generated by the CMS TYPO3
VS Code	✓	Electron : Chromium (a browser engine) + Node.js
Spotify	✓	Embedded Chromium; UI shared with the web player (React)
Steam	✓	Embedded Chromium; UI in React, shared with the Steam Deck
Claude Code	✓	TypeScript + React; Ink renders the UI as text in the terminal

See for yourself

VS Code is a web page in a desktop window. Open its dev tools:

Help → Toggle Developer Tools

- These are the same **Chrome dev tools** you'll use in Block 1
- Inspect any part of the editor: it's **HTML and CSS**

On Windows, open the Task Manager while Steam is running. The processes called **Steam Client WebHelper** are its embedded Chromium.

Websites, desktop apps, game stores, even terminal tools.

The web is **everywhere**.



What you learn here, you can use in almost every area of software.

PART 2

Software Development in the AI Age

Why learn this, if AI writes the code?

AI writes more and more of the code

AI can write...

- code
- tests
- build workflows
- automation scripts
- ...

You still have to...

- understand how code and systems are **structured**
- understand how they **work**
- judge whether the AI's result is **right**
- **build and maintain** the system — for years

This course teaches the part that stays with you: how the web works, from the HTTP request to the finished page.

“Everyone knows that debugging is twice as hard as writing a program in the first place. So if you're as clever as you can be when you write it, how will you ever debug it?”

— Brian Kernighan & P. J. Plauger, *The Elements of Programming Style* (1978)

Now the AI writes the code.

Who **debugs*** it?

You do. So you have to understand it.

PART 3

About This Course

What you'll learn, and how the course works

After this course, you can...

- Explain **how the web works**: client and server, URLs, HTTP
- Build web pages with **HTML** and **CSS**
- Program in **JavaScript** — in the browser and on the server (Node.js)
- Build a **server-rendered** web app (Express) and a **client-rendered** one (vanilla JavaScript, then Vue)
- **Compare** both architectures, and choose the right one for a task
- Understand the most common attacks on web apps (XSS, CSRF)

Topics & dates

Block	Topic	Week	Date
0	Intro	1	07.10.
1	Client/Server Model + HTTP	1	07.10.
2a–2b	HTML	2	14.10.
3a–3d	CSS	3–4	21.10.–28.10.
4	Styling with a framework (Bootstrap)	4	28.10.
5a–5b	JavaScript fundamentals	5	04.11.
6a–6b	DOM + Events & Interactivity	6	11.11.
–	<i>No lecture (Blockwoche)</i>	–	<i>18.11.</i>
7a–7b	Server-side rendering (SSR)	7	25.11.
8a–8d	Client-side rendering (CSR)	8–9	02.12.–09.12.
9	CSR with a framework (Vue)	10	16.12.
10	SSR vs CSR	10	16.12.
–	<i>No lecture (lecture-free period)</i>	–	<i>23.12.–06.01.</i>
–	Recap & questions	11	13.01.

When, where, how

WHEN

Wednesdays, **16:00–19:15** · 07.10.2026 to 13.01.2027

No lecture on 18.11.2026 (Blockwoche)

and from 23.12.2026 to 06.01.2027 (lecture-free period)

WHERE

Room **92.1.14**

A lab with Windows PCs. Your own laptop is welcome.

FORMAT

2 blocks per week

Each block: 45 min lecture + 45 min hands-on exercise

LANGUAGE

Slides and exercises in **English**

Lectures and exercises in German

Prerequisites, exam, contact

YOU BRING

Java from PG1 and PG2

Programming is not new to you — only the web is.

EXAM

Written exam, 90 minutes

QUESTIONS

In class,

in **Moodle**,

or by email matthias.kellnhofer@hs-ansbach.de

RECAP

Week 11 (13.01.2027) is for recap and your questions

Course material

github.com/mkellnhofer/web-development-course

- The link is also in **Moodle**
- **Week by week:** the material for a week comes out two days before the lecture (Monday)
- **Today is special:** Block 1 comes out right after exercise 0 — and you get it with Git
- Slides as **PDF**, exercise sheets as **Markdown** (GitHub and VS Code show them nicely)

How the material is organised

```
web-development-course/
├── slides/
│   ├── week01/ ...
│   └── week02/
│       ├── 2a-html-basics.pdf
│       └── 2b-html-forms.pdf
└── exercises/
    ├── week01/ ...
    └── week02/
        ├── 2a/
        │   ├── README.md ← sheet
        │   └── starter/ ← your code
        └── 2b/
```

- One folder per **week**: `weekNN/`
- Files start with the **block ID**:
`2a-html-basics.pdf` →
`exercises/week02/2a/`
- Block IDs follow the **topics**, as in the table
- Every slide shows where you are:
`week 02 > 2a > html-basics`

How to succeed in this course

In class

- **Lectures:** think, don't just listen
- **Exercises:** come every week, and do **all tasks yourself**
- **Ask** when something is unclear — or ask for help

At home

- Go through the **slides** again
- Do the **stretch goals** at the end of each exercise sheet
- Build **small apps** that interest you
- **Stay on track:** each block builds on the ones before

Your workload

5 ECTS = 150 hours in total

48 h in class

102 h self-study

≈ 6 hours per week outside class

Just like a foreign language:

You learn to program by **programming**.



Watching and reading are not enough. Write the code yourself.

Using AI in this course

Do the exercise yourself first.
Use AI to **explain**, not to **solve**.

✓ Helps you learn

- Explain an error message
- Explain code you don't understand
- Review your solution — after you're done

✗ Skips the learning

- Paste the task, copy the answer
- Keep code you can't explain

In the written exam, you work without AI. Only what you understand counts.

Literature & further reading

REFERENCE	MDN Web Docs (Mozilla) — HTML, CSS, JavaScript and Web APIs developer.mozilla.org
HTML & CSS	Shay Howe: Learn to Code HTML & CSS (New Riders) Free online: learn.shayhowe.com/html-css
JAVASCRIPT	Marijn Haverbeke: Eloquent JavaScript (No Starch Press) Free online: eloquentjavascript.net
MORE	Quick reference guides HTML: github.com/iLoveCodingOrg/html-cheatsheet CSS: github.com/iLoveCodingOrg/css-cheatsheet JavaScript: github.com/iLoveCodingOrg/javascript-cheatsheet

PART 4

Your Tools

What we use, and why

Four tools

VS Code

EDITOR

Write HTML, CSS and JavaScript. Free — and built with web technologies itself.

Chrome

BROWSER

Run your pages. Its dev tools show what happens inside.

Git

VERSION CONTROL

Get the course material, and keep track of your own work.

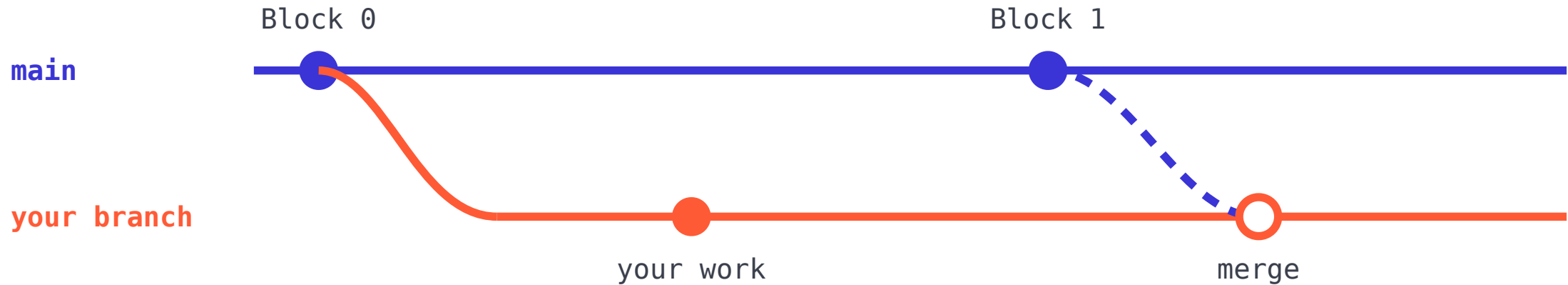
Node.js

JAVASCRIPT RUNTIME

Run JavaScript outside the browser — for tools and web servers.

You install all four in **exercise 0**. The exercise sheet has the steps.

Git: how you work with the course material



1. **Clone** the course repo
2. Create **your own branch** — all your work goes there
3. **New material?** Update `main`, then merge it into your branch

Git is really helpful — learn more of it on your own. In this course, this workflow is all you need.

Node.js & npm

- **Node.js** runs JavaScript outside the browser — on your laptop
- From Block 1 on, we use it to **serve** pages; from Block 7 on, you write your own **web servers**
- **npm** comes with Node: a package manager — think **Maven for Java**
- **npm** runs a package without installing it — `npm serve` serves a folder as a website

We use **Node 24** (LTS). You install it in exercise 0 and check it with `node --version`.

Your turn

Set up your tools — and get the course material with Git.

1. **VS Code** and **Chrome**
2. **Git**: install and set it up, **clone** the course repo, create **your own branch**, make your first **commit**
3. **Node.js 24**: install it, **serve** a page, change it, and commit the change
4. **Live**: when I publish Block 1, **get it** with Git

Sheet: you don't have the repo yet, so open the sheet on GitHub:

github.com/mkellnhofer/web-development-course → `exercises/week01/0/README.md`